Technical Design Document — PizzaMath

> Audience: Whole team (engineers + stakeholders) Last updated: 2026-04-15

---

1. Product Overview

One-line pitch: PizzaMath is a subscription-based K–12 math worksheet platform powered by AI generation and aligned to California Common Core Standards.

The problem it solves: Teachers and parents struggle to find grade-appropriate, standards-aligned math worksheets at the right difficulty level. Existing tools are generic; PizzaMath lets users request AI-generated variants (easier, harder, different format) on demand.

Who it's for: K–12 students, parents, and teachers in California.

Core Features

Feature	Description
Worksheet browsing	Filter by CA Common Core category, subcategory, level, and grade
AI worksheet generation	Admins use a multi-turn Claude chat to generate new worksheets with auto-populated metadata
Answer sheets	Every worksheet has a paired answer key
Progress tracking	Users record date, score, and comment per attempt
Subscription gate	All content requires an active $10/month or $100/year subscription
Admin panel	Full CRUD for worksheets + user management
File export	Download worksheets as PDF or DOC

What This Does NOT Do

- No real-time collaboration or classroom management
- No automated grading or student performance analytics
- No video or interactive content — worksheets only
- No payment processing (mocked in current implementation)

- No email notifications

Platform scope: Web application (desktop-first, responsive to tablet/mobile)

---

2. Tech Stack

Frontend

Layer	Technology	Version	Purpose
UI Framework	React	19.2.5	Component-based UI
Language	TypeScript	6.0.2	Type safety (strict mode)
Bundler	Vite	8.0.4	Dev server + production build
Routing	React Router DOM	7.14.0	Client-side routing
Server State	TanStack React Query	5.99.0	API data fetching + caching
Client State	Zustand	5.0.12	UI state (auth, filters)
Forms	React Hook Form	7.72.1	Controlled form handling
Validation	Zod	4.3.6	Schema validation + type inference
Styling	Tailwind CSS	4.2.2	Utility-first CSS
Class Utility	clsx + tailwind-merge	2.1.1 / 3.5.0	Conditional class composition via `cn()`
AI Integration	Anthropic SDK	0.88.0	Claude API streaming

Testing & Mocking

Layer	Technology	Version	Purpose
Test Runner	Vitest	4.1.4	Fast unit/integration tests
DOM Testing	Testing Library React	16.3.2	User-centric component tests
User Events	Testing Library User Event	14.6.1	Realistic user interaction simulation
DOM Matchers	Testing Library jest-dom	6.9.1	Extended DOM assertions
DOM Simulator	jsdom	29.0.2	Browser-like environment in Node
API Mocking	MSW	2.13.2	Network-level mock handlers
Mock DB	@mswjs/data	0.16.2	In-memory relational data store
Coverage	@vitest/coverage-v8	4.1.4	Code coverage reporting

Tooling

Tool	Version	Purpose
ESLint	9.39.4	Linting (flat config v9)
typescript-eslint	8.58.0	TypeScript-aware linting
eslint-plugin-react-hooks	7.0.1	Hooks rules enforcement

3. Architecture

Current Architecture (Mocked Backend)

PizzaMath is currently a fully client-side SPA. There is no real backend server — all API calls are intercepted at the network layer by **MSW (Mock Service Worker)**, which handles requests in the browser using an in-memory `@mswjs/data` store.

```
Browser
  │
  ├── React App (Vite SPA)
  │     ├── React Router v7 (client-side routing)
  │     ├── TanStack React Query (server state)
  │     ├── Zustand (auth + filter UI state)
  │     └── Anthropic SDK (direct Claude API calls)
  │
  └── MSW Service Worker (intercepts fetch)
        └── In-memory @mswjs/data store
              ├── Users, Categories, Subcategories
              ├── Worksheets + ProgressEntries
              └── Seed data (14 CC categories, 3 worksheets, 1 admin)
```

Request Flow

```
User Action
    │
    ▼
React Component
    │  calls hook
    ▼
React Query / Zustand
    │  calls api/ layer
    ▼
apiFetch() utility
    │  fetch() with Bearer token header
    ▼
MSW Service Worker (browser)
    │  matches route handler
    ▼
@mswjs/data in-memory DB
    │  returns mock response
    ▼
React Query cache → Component re-render
```

Source Tree

```
pizzamath/
├── public/
├── src/
│   ├── api/           ← typed fetch wrappers + Zod schemas + queryKeys
│   ├── components/    ← shared UI (Navbar, guards, layout)
│   ├── features/      ← auth, worksheets, progress, admin
│   ├── mocks/         ← MSW handlers + @mswjs/data schema + seed
│   ├── pages/         ← route-level components only
│   ├── stores/        ← Zustand (filter state)
│   ├── test/          ← Vitest global setup
│   ├── types/         ← shared global TypeScript types
│   ├── utils/         ← apiFetch, cn, logger
│   ├── App.tsx
│   ├── main.tsx       ← MSW init + QueryClientProvider
│   └── router.tsx     ← React Router config
├── CLAUDE.md
├── TECHNICAL_DESIGN.md
├── package.json
├── tsconfig.json
├── tsconfig.app.json
├── tsconfig.node.json
└── vite.config.ts
```

Auth Architecture

- Token = user ID string stored in `localStorage` via Zustand `persist` middleware (key: `pizzamath-auth`)
- Every API request attaches `Authorization: Bearer <token>` via the shared `apiFetch` wrapper
- Three guard components wrap routes: `AuthGuard` → `SubscriptionGuard` → `AdminGuard`
- No JWT signing in current implementation (mock env); production would use RS256-signed tokens

Production Migration Path

To move from mocked to real backend:

1. Replace MSW handlers with a real REST API (Node/Express, Go, etc.)
2. Replace `@mswjs/data` with a real database (PostgreSQL + Prisma recommended)
3. Move Claude API calls server-side (remove `dangerouslyAllowBrowser`)
4. Add real JWT signing + refresh token flow
5. Integrate Stripe for subscription billing

---

4. Database Schema

> **Note:** PizzaMath currently uses an in-memory `@mswjs/data` store (no persistent DB). The schema below reflects the current data model and is designed to map 1:1 to a future PostgreSQL implementation.

ER Overview

```
User ──────────────── ProgressEntry
 │                        │
 │                        │ worksheetId
 │                   Worksheet ─── Subcategory ─── Category
 │                                       │
 │                        (categoryId denormalized on Worksheet)
```

`USERS`

Column	Type	Constraints
id	string (UUID)	PK
email	string	UNIQUE, NOT NULL
password	string	NOT NULL (hashed in production)
role	enum('user','admin')	NOT NULL, DEFAULT 'user'
accountStatus	enum('active','suspended')	NOT NULL, DEFAULT 'active'
subscriptionStatus	enum('active','inactive','trial')	NOT NULL, DEFAULT 'inactive'
subscriptionPlan	enum('monthly','annual') | null	NULLABLE
subscriptionExpiresAt	ISO datetime | null	NULLABLE
createdAt	ISO datetime	NOT NULL

`CATEGORIES`

Column	Type	Constraints
id	string	PK (e.g., `cat-1`)
name	string	NOT NULL
grades	string	NOT NULL (e.g., `"K-5"`)

14 seeded categories (CA Common Core): `Counting & Cardinality`, `Operations & Algebraic Thinking`, `Number & Operations in Base Ten`, `Number & Operations – Fractions`, `Measurement & Data`, `Geometry`, `Ratios & Proportional Relationships`, `The Number System`, `Expressions & Equations`, `Functions`, `Statistics & Probability`, `Number & Quantity`, `Algebra`, `Modeling`

`SUBCATEGORIES`

Column	Type	Constraints
id	string	PK (e.g., `sub-1-1`)
name	string	NOT NULL
categoryId	string	FK → categories.id, NOT NULL

`WORKSHEETS`

Column	Type	Constraints
id	string (UUID)	PK
title	string	NOT NULL
categoryId	string	FK → categories.id, NOT NULL
subcategoryId	string	FK → subcategories.id, NOT NULL
level	enum('Beginner','Intermediate','Advanced')	NOT NULL
schoolGrade	enum('K','1'–'12') | null	NULLABLE
author	string	NOT NULL
content	text (markdown)	NOT NULL
answerContent	text (markdown)	NOT NULL
createdAt	ISO datetime	NOT NULL

`PROGRESS_ENTRIES`

Column	Type	Constraints
id	string (UUID)	PK
userId	string	FK → users.id, NOT NULL
worksheetId	string	FK → worksheets.id, NOT NULL
worksheetTitle	string	NOT NULL (denormalized)
date	string (YYYY-MM-DD)	NOT NULL
score	number (0–100)	NOT NULL
comment	string	NOT NULL (empty string if none)

Indexes Summary

Table	Index	Type
users	email	UNIQUE
subcategories	categoryId	INDEX
worksheets	categoryId, subcategoryId	COMPOSITE INDEX
worksheets	level	INDEX
progress_entries	userId	INDEX
progress_entries	worksheetId	INDEX

5. API Specification

Base URL: `/api`
Auth: `Authorization: Bearer <token>` (token = user ID in mock; JWT in production)
Content-Type: `application/json` (except file export which returns `application/octet-stream`)

Auth

Method	Path	Auth	Description
POST	`/auth/login`	None	Email + password → `{token, user}`
POST	`/auth/register`	None	Email + password + plan → `{token, user}`
POST	`/auth/logout`	Yes	Clears session → `{ok: true}`
GET	`/auth/me`	Yes	Returns current `User`
POST	`/auth/change-password`	Yes	`{currentPassword, newPassword}` → `{ok: true}`

Worksheets

Method	Path	Auth	Description
GET	`/worksheets`	Yes	List worksheets. Query: `?categoryId=&subcategoryId=&keyword=`
GET	`/worksheets/:id`	Yes	Get single worksheet with content + answerContent
POST	`/worksheets`	Admin	Create worksheet. Body: `WorksheetFormInput`
DELETE	`/worksheets/:id`	Admin	Delete worksheet
GET	`/worksheets/:id/export`	Yes	Export. Query: `?format=pdf|doc`. Returns blob

> **Export note:** Current implementation returns a mock blob. Production will require server-side PDF generation (e.g., Puppeteer, WeasyPrint) or a client-side library (react-pdf).

Categories

Method	Path	Auth	Description
GET	`/categories`	Yes	Returns all categories, each with a nested `subcategories` array

Progress

Method	Path	Auth	Description
GET	`/progress`	Yes	Query: `?userId=`. Returns user's `ProgressEntry[]`
POST	`/progress`	Yes	Body: `{worksheetId, date, score, comment}` → new `ProgressEntry`

Users (Admin)

Method	Path	Auth	Description
GET	`/users`	Admin	List all users
POST	`/users`	Admin	Create user. Body: `{email, password, role, plan}`
PATCH	`/users/:id`	Admin	Update user (e.g., suspend). Body: `{accountStatus?}`
DELETE	`/users/:id`	Admin	Delete user

6. Frontend Component Structure

Routes

Route	Page	Auth	Subscription	Admin
`/login`	LoginPage	No	No	No
`/register`	RegisterPage	No	No	No
`/subscribe`	SubscribePage	No	No	No
`/`	BrowsePage	Yes	Yes	—
`/worksheets/:id`	WorksheetPage	Yes	Yes	—
`/account`	AccountPage	Yes	Yes	—
`/history`	UsageHistoryPage	Yes	Yes	—
`/admin`	AdminPage	Yes	Bypassed	Yes
`/admin/generate`	GeneratePage	Yes	Bypassed	Yes

Pages

Page	Responsibility
LoginPage	Email/password login form
RegisterPage	Registration + subscription plan selection
SubscribePage	Prompt inactive subscribers to subscribe
BrowsePage	Worksheet listing with category/subcategory/keyword filtering
WorksheetPage	Full worksheet view, answer sheet toggle, export, progress logging
AccountPage	Change password, view subscription status
UsageHistoryPage	User's progress history table
AdminPage	Admin dashboard: worksheet CRUD + user management
GeneratePage	Multi-turn Claude chat → worksheet generation

Shared Components (`src/components/`)

Component	Purpose
AppLayout	Page shell with Navbar
Navbar	Top bar: logo, category dropdowns, search, account menu
CategoryDropdown	Controlled CA Common Core category selector
SubcategoryDropdown	Conditional subcategory selector (depends on selected category)
SearchBar	Keyword + optional category filter
AccountMenu	Dropdown: account link, history link, logout
AuthGuard	Redirects unauthenticated users to `/login`
SubscriptionGuard	Redirects inactive subscribers to `/subscribe` (admins bypass)
AdminGuard	Redirects non-admins to `/`

Feature Modules (`src/features/`)

auth/

- `LoginForm`, `RegisterForm`, `ChangePasswordForm` — form components
- `useAuth` — login/register/logout mutations + current user query

- `useSubscriptionGate` — subscription status check
- `store.ts` — Zustand auth store (user + token, persisted)

worksheets/

- `WorksheetCard` — preview card in browse grid
- `WorksheetViewer` — full content + answer sheet toggle
- `ExportButton` — PDF/DOC download trigger
- `useWorksheets` — filtered worksheet list query

progress/

- `ProgressEntryForm` — record date/score/comment for an attempt
- `ProgressHistory` — table of past attempts
- `useProgress` — progress entries query + create mutation

admin/

- `AdminDashboard` — hub with tabs for worksheets and users
- `GenerationChat` — multi-turn streaming chat with Claude; "Generate" button parses metadata and populates form
- `WorksheetForm` — manual worksheet creation/edit form
- `UserManagement` — user list with suspend/delete actions
- `AddUserForm` — admin-created user form
- `useGenerationSession` — ephemeral Claude conversation state
- `useAdminWorksheets` — worksheet CRUD mutations
- `useUserManagement` — user CRUD mutations

Where to Put a New Component

> - **Reused across features?** → `src/components/`
> - **Used only in one feature?** → `src/features/<feature>/components/`
> - **Route-level entry point?** → `src/pages/`
> - **Folder structure:** one component per folder with co-located hook, types, and test

State Management Summary

State	Tool	Store / Location
Authenticated user + token	Zustand (persisted)	`features/auth/store.ts`
Active category/subcategory/keyword filters	Zustand (ephemeral)	`stores/filterStore.ts`
API server data (worksheets, categories, progress, users)	React Query	via hooks in `features/*/hooks/`
Generation chat history	Local `useState`	`useGenerationSession.ts`
Form state	React Hook Form	inside form components

7. Testing Strategy

Three-Layer Model

Layer	Tool	Location	What It Tests
Unit	Vitest	`src/utils/*.test.ts`, `src/features/*/hooks/*.test.ts`	Pure functions, custom hooks
Integration	Vitest + RTL + MSW	`src/features/*/components/*.test.tsx`, `src/pages/*.test.tsx`	Components with real network mock
E2E	(planned) Playwright	`e2e/`	Full user flows in browser

Philosophy

Unit tests cover pure utilities (`cn`, `logger`, `apiFetch` error paths) and hook logic in isolation.

Integration tests are the primary layer. They render components inside a full `QueryClientProvider` + MSW server, exercise real user interactions via `@testing-library/user-event`, and assert on visible DOM output. This matches how users actually experience the app.

No snapshot tests. Snapshots couple tests to markup details rather than behavior; they fail on irrelevant changes and give false confidence.

Mock at the network boundary using MSW — never mock modules directly. This ensures the full `api/` layer (Zod parsing, error handling, request headers) is exercised in every test.

TDD Workflow

1. Write a failing test describing the behavior
2. Run `npm run test` — confirm it fails
3. Implement the minimum code to make it pass
4. Refactor, keeping tests green

Test Commands

```
npm run test                                          # run all tests (watch mode)
npm run test -- --run                                 # run once (CI mode)
npm run test -- --reporter=verbose src/path/to/file.test.tsx  # single file
npm run test -- --coverage                            # with coverage report
npm run typecheck                                     # tsc --noEmit
npm run lint                                          # ESLint
```

Coverage Targets

Area	Target
Utilities (`src/utils/`)	100%
API layer (`src/api/`)	90%
Feature hooks (`src/features/*/hooks/`)	85%
Feature components	80%
Pages	70%

8. Accessibility Strategy

Target: WCAG 2.1 Level AA

Math education tools serve students with a range of learning needs. Accessibility is not optional.

Principles

Principle	Implementation
Perceivable	Sufficient color contrast (≥4.5:1 text, ≥3:1 UI), no color-only meaning, alt text on images
Operable	Full keyboard navigation, visible focus indicators, no keyboard traps
Understandable	Clear error messages at field level, consistent navigation, descriptive labels
Robust	Semantic HTML, ARIA roles only where native HTML falls short

Implementation Checklist

Semantic HTML

- [ ] Use `<nav>`, `<main>`, `<header>`, `<section>` landmarks
- [ ] Heading hierarchy: one `<h1>` per page, logical `h2` / `h3` nesting
- [ ] Lists for grouped items (`<ul>`, `<ol>`)

Forms

- [ ] Every input has a visible `<label>` linked via `for` / `id` or `aria-labelledby`
- [ ] Field-level error messages use `aria-describedby`
- [ ] Required fields marked with `aria-required="true"`
- [ ] Success/error states announced via `aria-live="polite"`

Interactive Components

- [ ] Dropdowns use native `<select>` or proper ARIA combobox pattern
- [ ] Modals/dialogs trap focus and restore on close
- [ ] Buttons vs links used semantically (buttons = action, links = navigation)
- [ ] Focus visible on all interactive elements (Tailwind `focus-visible:ring`)

Color & Contrast

- [ ] Text contrast ≥4.5:1 against background
- [ ] Interactive element contrast ≥3:1
- [ ] Error/success states not indicated by color alone

Testing Approach

Type	Tool	When
Automated	`@testing-library/jest-dom` + RTL's `getByRole` / `getByLabelText` queries	Every test (queries fail if ARIA roles are wrong)
Automated	axe-core (via `@axe-core/react`)	Dev mode — logs violations to console
Manual	Keyboard-only navigation walkthrough	Before each major release
Manual	Screen reader (VoiceOver on macOS)	Before each major release

9. Performance & Scalability

Frontend Performance

Area	Approach
Bundle splitting	Vite automatic code-splitting by route (React Router lazy loading)
Caching	React Query `staleTime: 5min` — category list and worksheet list cached aggressively
Image optimization	Worksheets are text/markdown — no heavy image assets currently
Tailwind CSS	Vite plugin purges unused classes at build time
Claude streaming	Token-by-token streaming for generation chat — no blocking wait for full response

Backend Performance (Production Target)

Area	Approach
Worksheet queries	Index on `(categoryId, subcategoryId)` + `level` for filter performance
Progress queries	Index on `userId` — each user only sees their own data
Auth	Stateless JWT — no server-side session storage needed
Claude API caching	Ephemeral prompt caching on system message (~5min TTL, ~90% token savings on repeated calls)

Scalability Constraints

Constraint	Threshold	Migration Path
In-memory mock DB	Dev/test only — no production use	Replace with PostgreSQL + Prisma
Claude API calls from browser	`dangerouslyAllowBrowser: true` (demo only — exposes API key)	Move to backend proxy endpoint
Subscription billing	Mocked	Integrate Stripe Checkout + webhooks
File export (PDF/DOC)	Mock blob response	Server-side Puppeteer or react-pdf for real files

Caching Strategy

Data	Cache	TTL
Category list	React Query	5 min (rarely changes)
Worksheet list	React Query	5 min
Single worksheet	React Query	5 min
Current user (`/auth/me`)	React Query	Until logout
Progress entries	React Query	2 min (updated frequently)
Claude system prompt	Anthropic ephemeral cache	~5 min per session

10. Responsiveness Strategy

Breakpoints (Tailwind defaults)

Prefix	Min Width	Target Devices
(none)	0px	Mobile (portrait)
`sm:`	640px	Mobile (landscape), small tablet
`md:`	768px	Tablet (portrait)
`lg:`	1024px	Tablet (landscape), laptop
`xl:`	1280px	Desktop

Layout Strategy

Section	Mobile	Tablet	Desktop
Navbar	Stacked: logo + hamburger menu	Logo + category dropdowns + condensed search	Full: logo + dropdowns + search + account
Browse grid	1 column	2 columns	3–4 columns
Worksheet viewer	Full width, single column	Full width	Centered, max-width container with sidebar for actions
Admin panel	Tabs stack vertically	Side-by-side tabs + content	Two-column layout
Generation chat	Full width, scrollable	Full width	Centered chat + form panel side by side
Forms (login/register)	Full width	Centered card, max-width 480px	Same as tablet

Implementation Rules

- Mobile-first: write base styles for mobile, use `md:` / `lg:` to override for larger screens
- No fixed pixel widths — use `max-w-*`, `w-full`, and `flex` / `grid` for fluid layouts
- Touch targets: minimum `44×44px` for all interactive elements (`min-h-11 min-w-11`)
- Horizontal scrolling: never allowed on any page at any breakpoint
- Tables: on mobile, use horizontal scroll inside a `overflow-x-auto` container

Typography Scaling

Element	Mobile	Desktop
Page title (`h1`)	`text-2xl`	`text-4xl`
Section heading (`h2`)	`text-xl`	`text-2xl`
Body	`text-sm`	`text-base`
Labels / meta	`text-xs`	`text-sm`